IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Datos listos para el modelo

Separar, escalar, codificar y encadenar con un **Pipeline**

Sprint 6 · S06_03 · 40 min en clase (ejercicios 2, 4 y 5)

IDEA GENERAL

Tres problemas, tres herramientas

Preprocesar es dejar la tabla lista para el modelo: completa, en escalas parecidas y solo con números.

Hay huecos	`SimpleImputer`	**Imputar**: rellenar los vacíos, por ejemplo con la mediana.
Escalas distintas	`StandardScaler`	**Escalar**: un precio de 60 a 140 pesa más que los niños, de 0 a 3.
Texto, no números	`OneHotEncoder`	**Codificar**: convertir categorías, como la zona, en números.

SEPARAR ENTRENAMIENTO Y PRUEBA · EJERCICIO 1 (CASA)

Guardar un examen sorpresa

fila	0	1	2	3	4	5	6	7	8	9
cancelada (y)	0	0	0	1	0	1	0	0	0	1
va a	train	**test**	train	train	train	train	**test**	train	train	**test**

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=7)
len(X_train), len(X_test)        # 7 3
```

test_size=0.3

El **30 %** queda como prueba: el modelo nunca lo ve al aprender.

stratify=y

Canceladas en proporción casi igual en ambos trozos: 2 de 7 y 1 de 3.

random_state=7

Fija el azar: el mismo reparto cada vez que se ejecuta.

ESCALADO · EJERCICIO 2

Poner las columnas en la misma escala

Sin escalar, el modelo se fija en la columna de números grandes: el precio (60 a 140) pesaría más que los niños (0 a 3).

Precio (€)	MinMax	Standard
60	0	−1,18
80	0,25	−0,51
100	0,5	0,17
140	1	1,52

MinMaxScaler

Todo entre **0 y 1**: (x − mínimo) / (máximo − mínimo).

StandardScaler

Media 0 y desviación 1: (x − media) / desviación. Aquí, media 95 y desviación 29,58.

Si entre los precios hay un 999 €, MinMax aplasta a los demás: 60, 80, 100 y 140 quedan entre 0 y 0,09.

ESCALADO · EJERCICIO 2

Aprender solo del entrenamiento

fit = aprender

El escalador mira los datos y apunta lo que necesita.

transform = aplicar

Usa lo apuntado para convertir valores.

```
# X_train: 60, 80, 100, 140   ·   X_test: 100, (vacío), 160
mediana = X_train["precio"].median()                    # 90.0
esc = MinMaxScaler().fit(X_train[["precio"]])           # mínimo 60, máximo 140
esc.transform(X_test[["precio"]].fillna(mediana))       # 0.5 0.375 1.25
```

El 1,25 es normal: el 160 € supera lo visto al aprender. Con fit_transform en la prueba saldría 0,14 · 0 · 1: el escalador se reajustaría con ella.

CODIFICACIÓN · EJERCICIO 3 (CASA)

Si hay orden: ordinal

Los modelos solo entienden números: hay que **codificar** las categorías. Si tienen un **orden real** (baja, media, alta), se numeran en ese orden.

satisfaccion	Código
media	1
alta	2
baja	0
media	1

```
orden = [["baja", "media", "alta"]]   # una lista por columna
ord_sat = OrdinalEncoder(categories=orden)
ord_sat.fit_transform(df[["satisfaccion"]])   # 1, 2, 0, 1
```

Sin **categories**, el orden sería alfabético: alta 0, baja 1, media 2. **Un orden inventado.**

CODIFICACIÓN · EJERCICIO 4

Sin orden: una columna por categoría

zona	zona_norte	zona_oeste	zona_sur
norte	1	0	0
sur	0	0	1
norte	1	0	0
oeste	0	1	0

¿Por qué no 1, 2, 3?

Sur (3) sería «más» que **norte (1)**: un orden falso.

handle_unknown="ignore"

Si en la prueba aparece una zona nueva (este), todas sus columnas valen **0** en vez de dar error.

```
oh = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
oh.fit_transform(df[["zona"]])                  # la tabla de arriba
oh.transform(pd.DataFrame({"zona": ["este"]}))  # 0, 0, 0 (zona nueva)
```

COLUMNTRANSFORMER · EJERCICIO 4

Cada tipo de columna, su receta

```
prep = ColumnTransformer([       # cada rama: (nombre, qué hacer, columnas)
    ("num", Pipeline([("imputar", SimpleImputer(strategy="median")),
                      ("escalar", StandardScaler())]), ["precio"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["zona"])])
# el precio vacío se rellena con la mediana (100) y al escalar queda en 0
```

precio	zona	num__precio	cat__zona_norte	cat__zona_oeste	cat__zona_sur
60	norte	−1,41	1	0	0
vacío	sur	0	0	0	1
100	norte	0	1	0	0
140	oeste	1,41	0	1	0

PIPELINE · EJERCICIO 5

Una cadena de montaje para el modelo

```
modelo = Pipeline([("prep", prep),
                   ("clf", LogisticRegression(max_iter=1000))])
modelo.fit(X_train, y_train)      # prep aprende SOLO de train; luego el modelo
modelo.score(X_test, y_test)      # 0.75: acierta 3 de 4
```

fit

Cada paso aprende **solo** de entrenamiento y pasa el resultado al siguiente.

score

La prueba atraviesa los mismos pasos, **sin reajustar** nada. Devuelve la **exactitud**: proporción de aciertos.

12 reservas de ejemplo (8 para entrenar y 4 para probar); **prep** es el de la diapositiva anterior. Referencia: decir siempre «lo más frecuente» acertaría 2 de 4 (0,5).

FUGA DE DATOS · SECCIÓN 6 (CASA)

Cuando la prueba se cuela al aprender

Fuga de datos: el modelo «ve» información de la prueba mientras aprende, y su nota sale inflada.

8 precios: entrenamiento 60, 80, 100, 100, 120, 140 · prueba 180 y 300.

MAL: FIT CON TODOS

```
esc = StandardScaler().fit(todos)
esc.mean_[0]             # 135.0
esc.transform([[300]])   # 2.32
```

BIEN: FIT SOLO CON TRAIN

```
esc = StandardScaler().fit(train)
esc.mean_[0]             # 100.0
esc.transform([[300]])   # 7.75
```

El escalador «mal» ya conocía el 300 y lo ve casi normal (2,3 desviaciones). **Pipeline** lo evita: solo ajusta con lo que recibe **fit**.

MINI-RETO · REPRODUCIBILIDAD (CASA)

¿Mismas predicciones al repetirlo?

```
modelo2 = clone(modelo)                # copia nueva, sin entrenar
modelo2.fit(X_train, y_train)
pred1 = modelo.predict(X_test)         # [0 0 1 0]
pred2 = modelo2.predict(X_test)        # [0 0 1 0]
np.array_equal(pred1, pred2)           # True
```

clone

Crea un pipeline **nuevo y sin entrenar**, con la misma configuración.

np.array_equal

Compara dos listas: **True** si son **idénticas**.

Por qué importa

Quien repita tu trabajo, en otro ordenador, debe obtener lo mismo.

modelo, **X_train** y **X_test** son los del ejemplo del pipeline.

HOY HAS APRENDIDO

Una receta única y sin fuga

Separar

train_test_split con **stratify** y semilla

Transformar

Imputar, escalar y codificar: **fit** solo con entrenamiento

Encadenar

ColumnTransformer + **Pipeline**: una receta, sin fuga

Siguiente: S06_04 · métricas y validación.